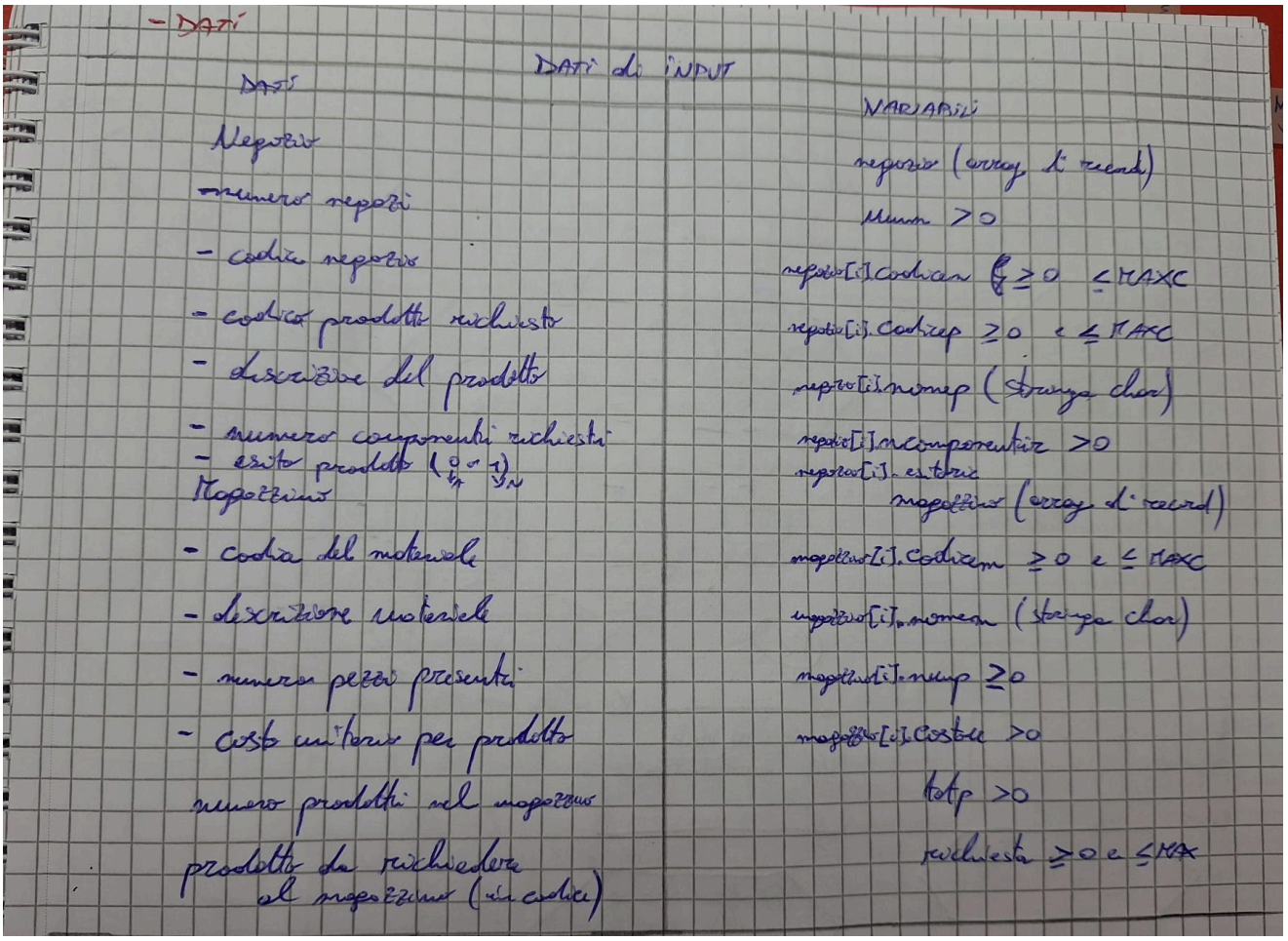
BRICOLAGE

PUNTO A

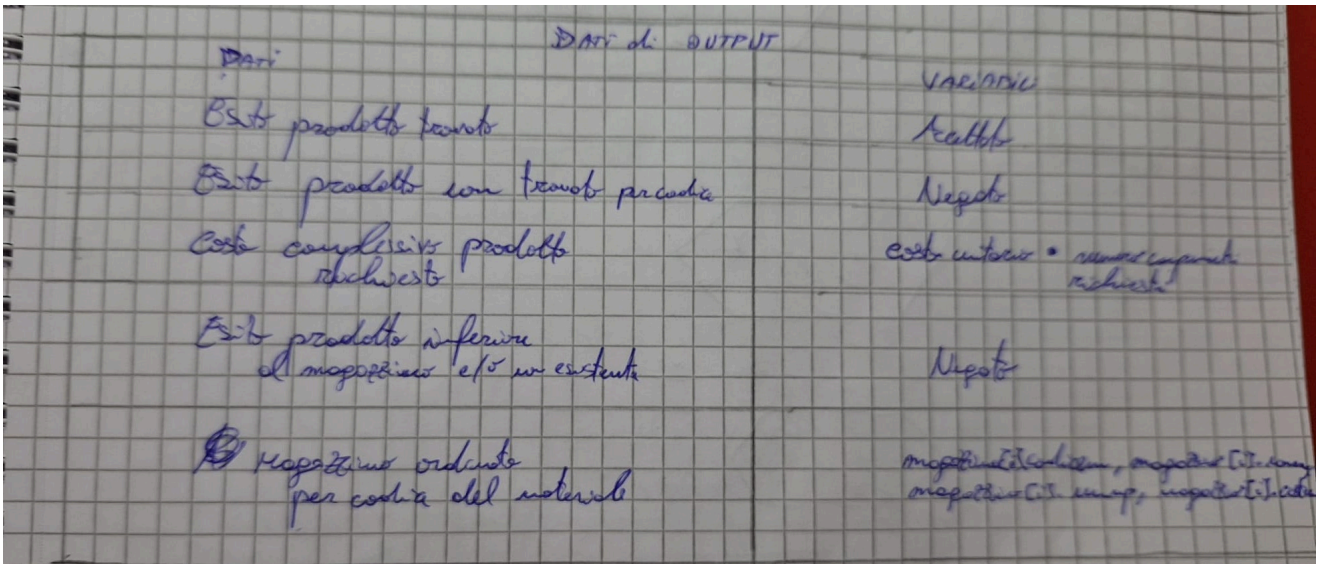
CHIARIFICA:

Creare un programma che detenga le informazioni di un negozio di bricolage ed il suo magazzino, i componenti del negozio sono gestiti in un array di record, tra cui il codice del prodotto, il codice del negozio, la descrizione del prodotto ed il numero dei componenti richiesti al magazzino per ogni tipo di prodotto. Il magazzino detiene in un altro array di record le informazioni del codice del materiale, la descrizione, il numero dei pezzi presenti nel magazzino e il costo unitario per ogni prodotto. Il programma deve ricevere in input una richiesta di acquisizione del prodotto dal magazzino e restituire in output un messaggio della richiesta al prodotto che sia in stato di accettata o negata. Si ipotizza che la risposta sia accettata, attraverso dei controlli, se il codice del prodotto corrisponde al codice del materiale e che il magazzino detenga delle copie maggiori alla richiesta del negozio, in tutti gli altri casi sarà negata. Si ipotizza inoltre che i codici siano solo numerici, fino ad un massimo di 5 cifre e che l'utente sappia che non ci sono codici uguali per prodotti diversi, per cui non sarà eseguito alcun controllo per esso. Inoltre si richiede di mostrare il costo complessivo del prodotto richiesto ove la richiesta sia accettata, riportando i corretti valori tra le funzioni. Il codice in fine dovrà ordinare l'array del magazzino per il codice del materiale. Si ipotizza che l'ordinamento venga fatto in ordine crescente in un apposita funzione, tramite algoritmo di bubble-sort.
Si ipotizza inoltre che la descrizione del prodotto sia il nome di quest'ultimo e che il codice del negozio possa pur essere il medesimo e non diverso poiché lo stesso negozio può inviare più richieste di materiali quanto le altre catene, con cui condividono lo stesso deposito.
Si ipotizza che non vengano effettuati controlli se nel magazzino siano presenti prodotti uguali.

DATI INPUT:

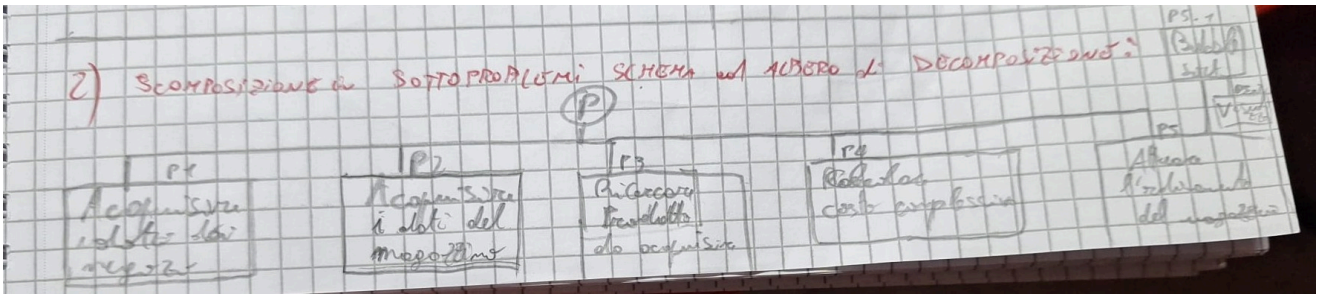


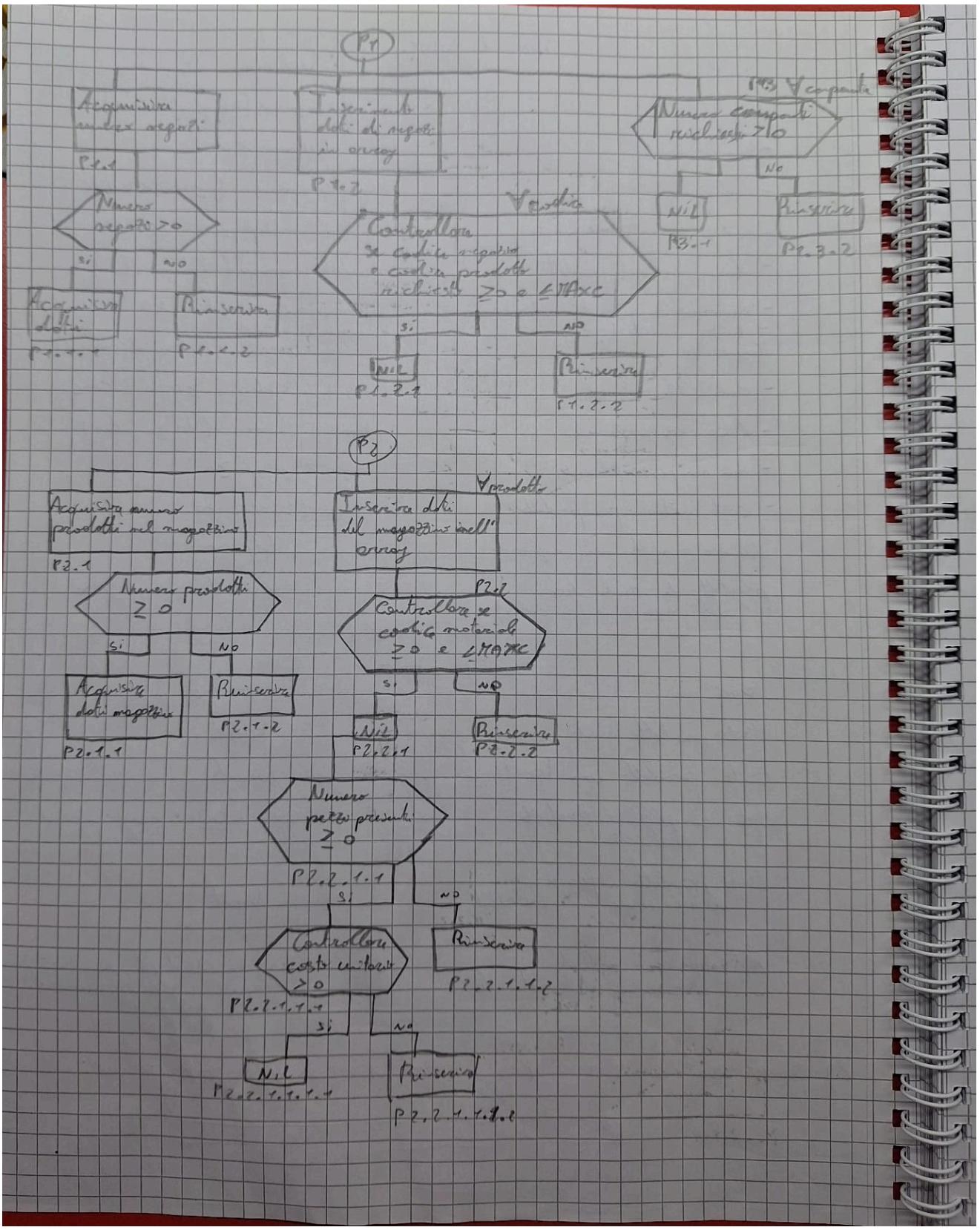
DATI OUTPUT:

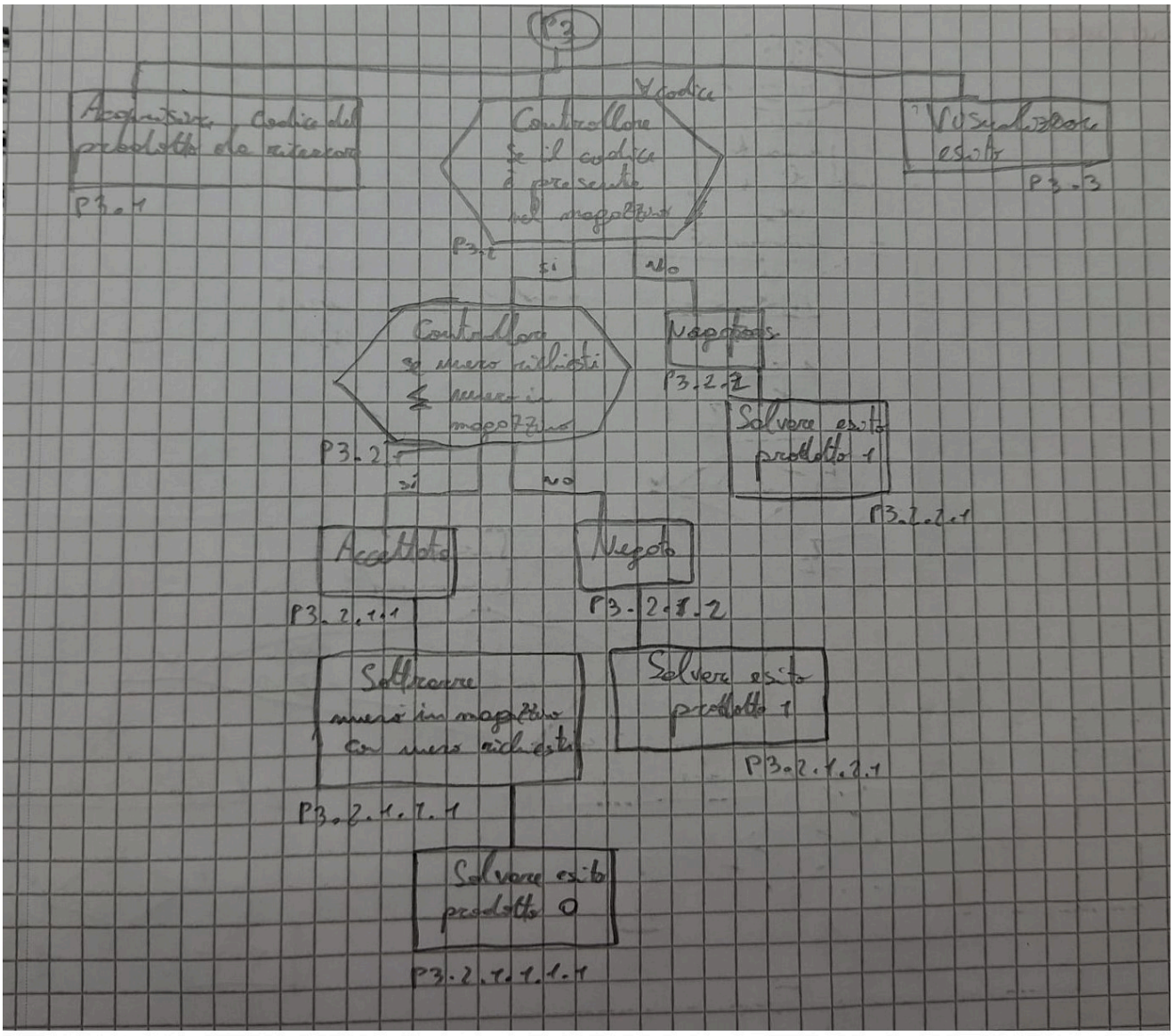


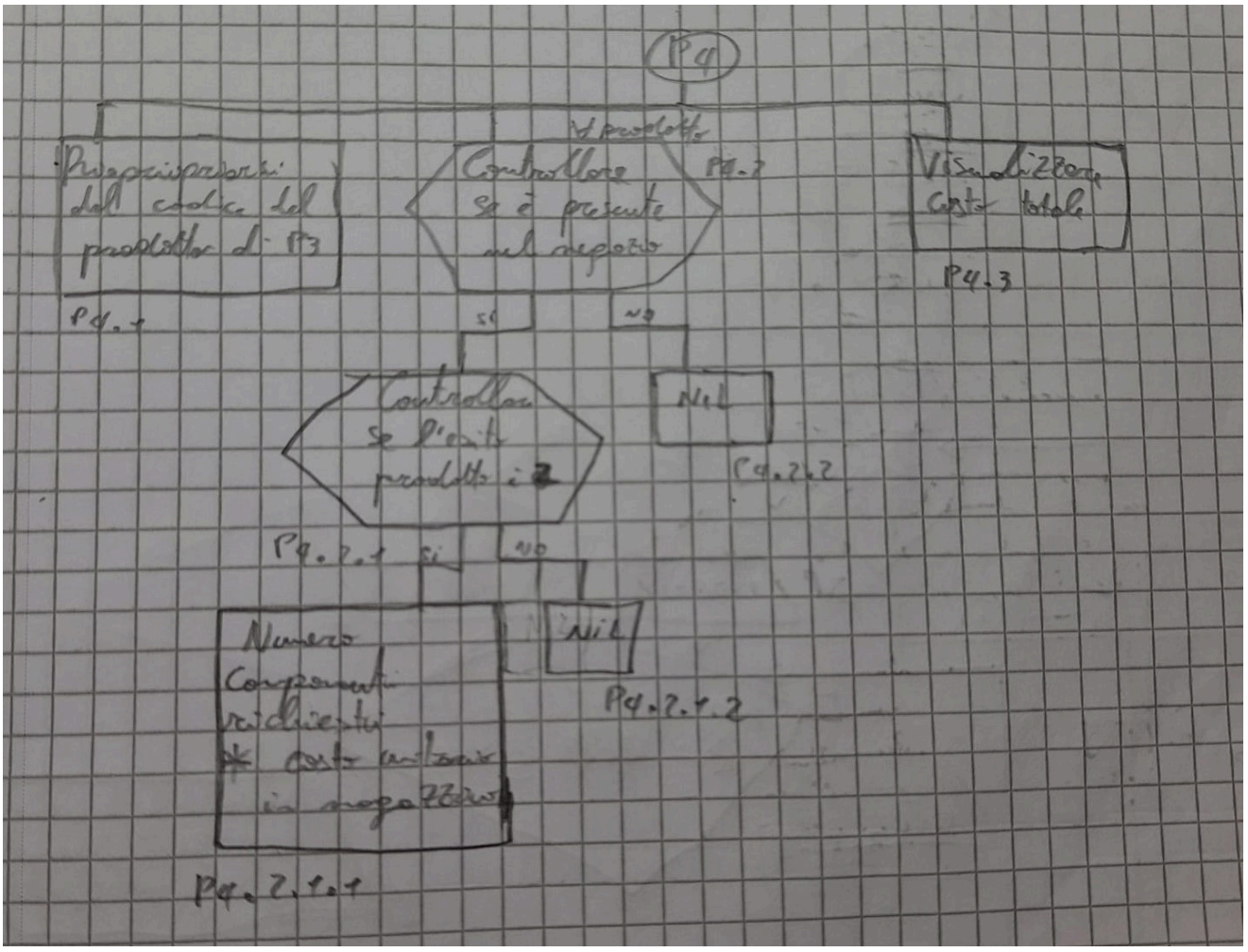
PUNTO B

ALBERO DI DECOMPOSIZIONE IN SOTTOPROBLEMI:


2) SCOMPOSIZIONE in SOTTOPROBLEMI SCHEMA ad ALBERO di DECOMPOSIZIONE:
P
P1
Acquisire i dati dei negozi
P2
Acquisire i dati del magazzino
P3
Ricercare prodotto da acquisire
P4
Calcolare costo complessivo
P5
Aggiornare disponibilità del magazzino


P1
Acquisire index negozi
P1.1
Inserimento dati di negozi in array
P1.2
P3 ∀ compatto
Numero compatti richiesti > 0
No
NIL
P3.1
Riinserire
P2.3.2
Numero negozi > 0
Si
No
Acquisire dati
P1.1.1
Riinserire
P1.1.2
∀ codice
Controllare se codice negozio e codice prodotto richiesti ≥ 0 e ≤ MAXC
Si
No
NIL
P1.2.1
Riinserire
P1.2.2
P2
Acquisire numero prodotti nel magazzino
P2.1
Inserire dati del magazzino nell' array
∀ prodotto
P2.2
Numero prodotti ≥ 0
Si
No
Acquisire dati magazzino
P2.1.1
Riinserire
P2.1.2
Controllare se codice materiale ≥ 0 e ≤ MAXC
Si
No
NIL
P2.2.1
Riinserire
P2.2.2
Numero pezzi presenti ≥ 0
P2.2.1.1
Si
No
Controllare costo unitario > 0
P2.2.1.1.1
Riinserire
P2.2.1.1.2
Si
No
NIL
P2.2.1.1.1.1
Riinserire
P2.2.1.1.1.2

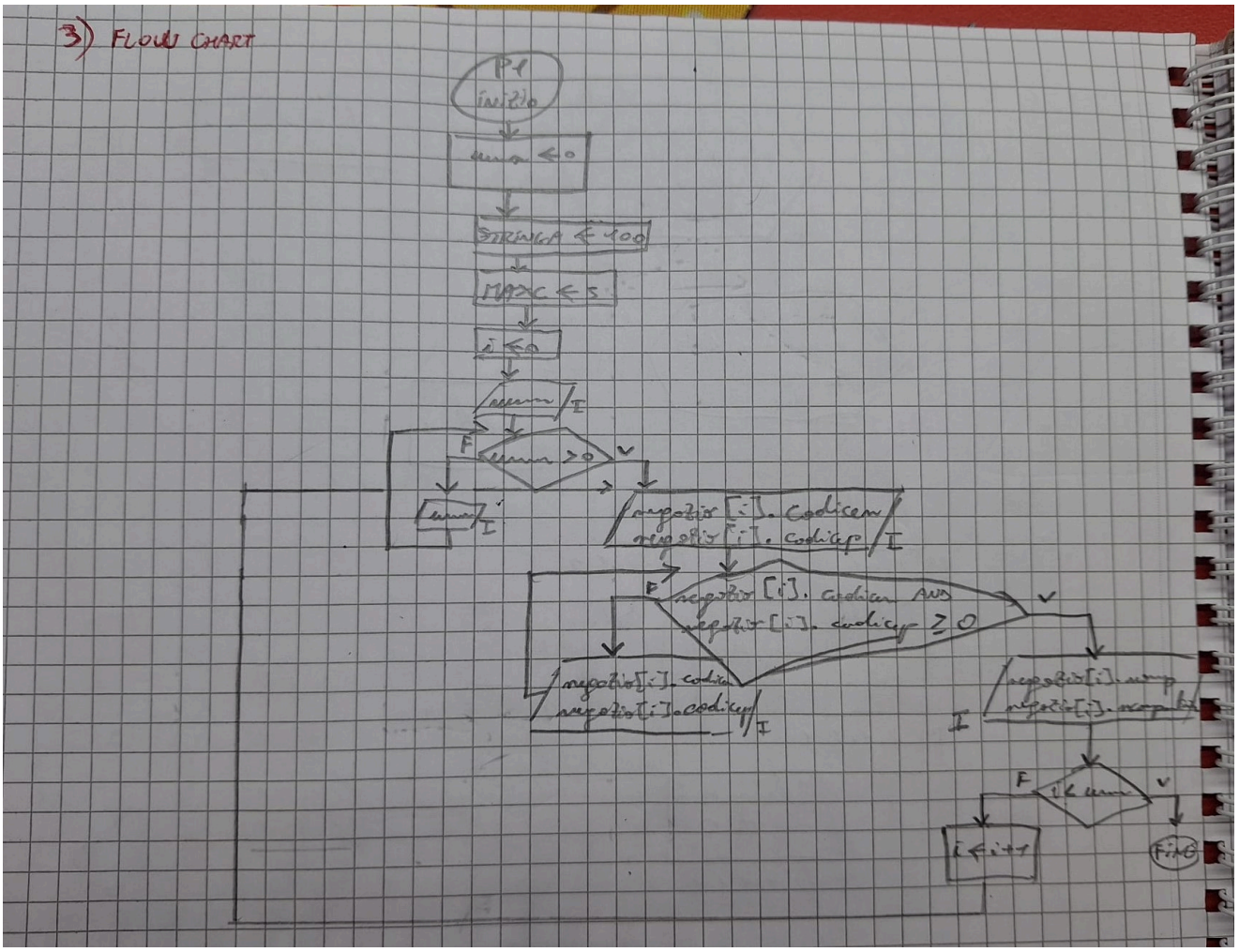

P3
Acquisizione codice del prodotto da ritirare
P3.1
Controllare se il codice è presente nel magazzino
P3.2
Visualizzare esito
P3.3
si
no
Controllare se numero richiesti ≤ numero in magazzino
P3.2.1
Negato
P3.2.2
Salvare esito prodotto 1
P3.2.2.1
si
no
Accettato
P3.2.1.1
Negato
P3.2.1.2
Sottrarre numero in magazzino con numero richiesto
P3.2.1.1.1
Salvare esito prodotto 1
P3.2.1.2.1
Salvare esito prodotto 0
P3.2.1.1.1.1

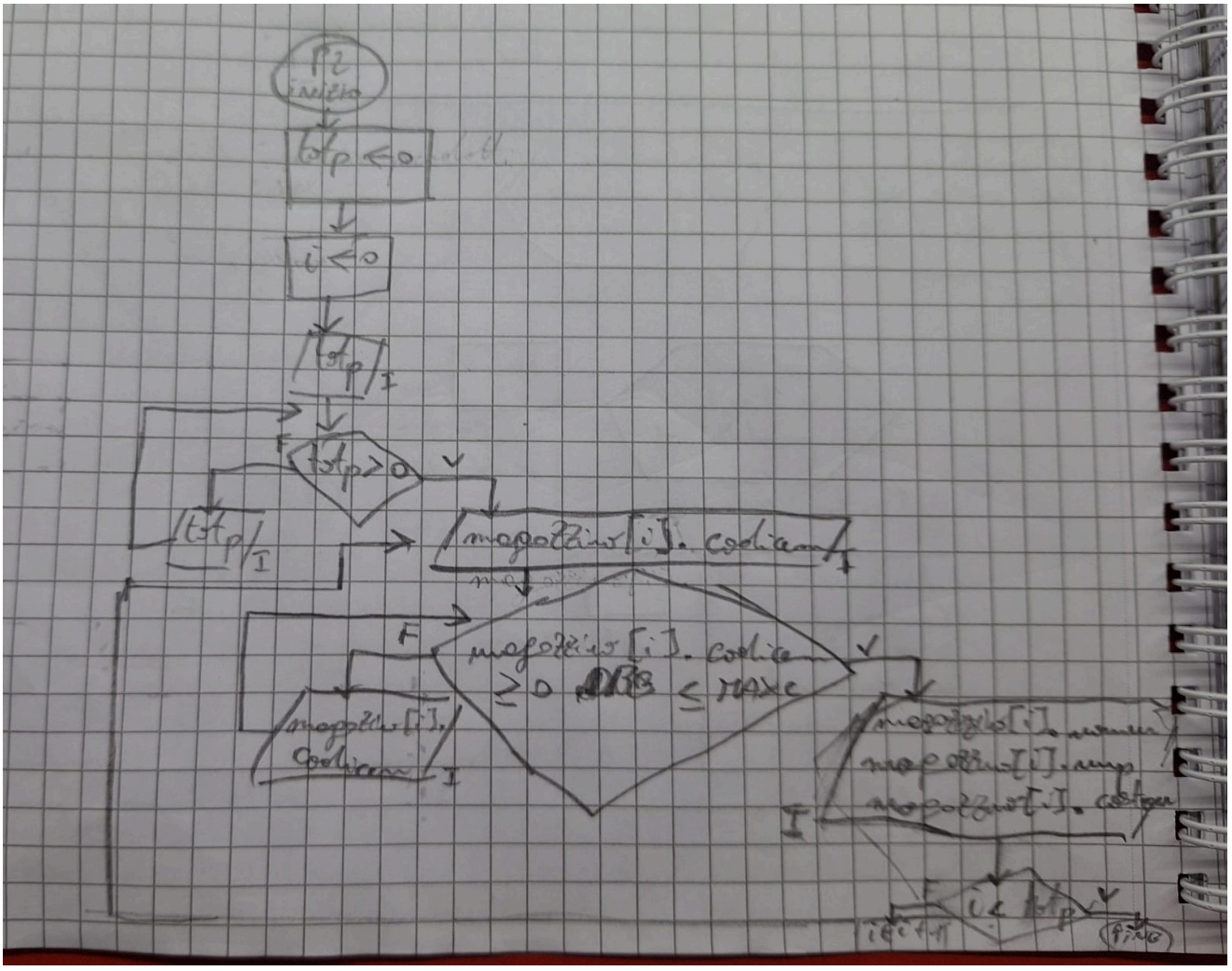


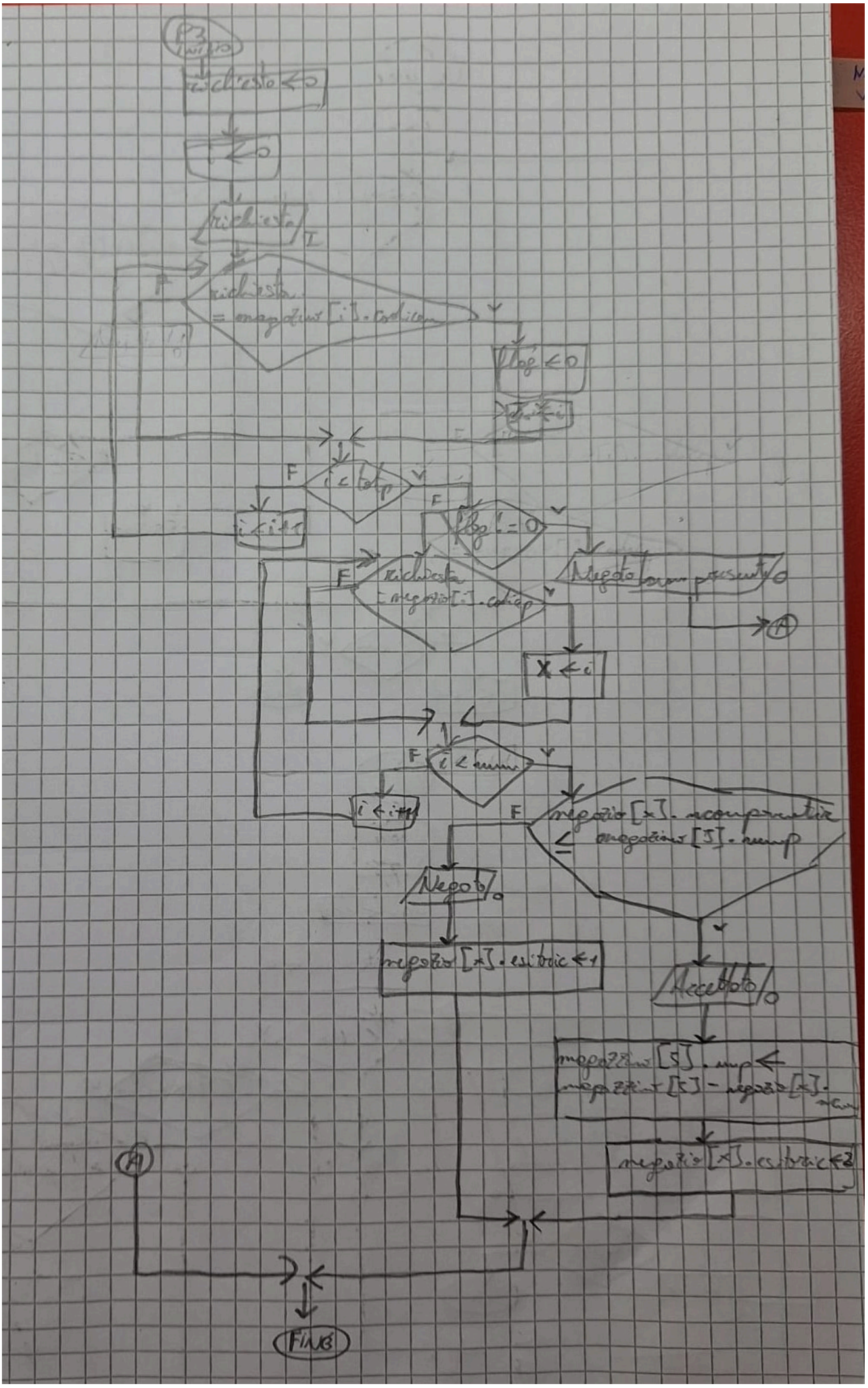
PUNTO C

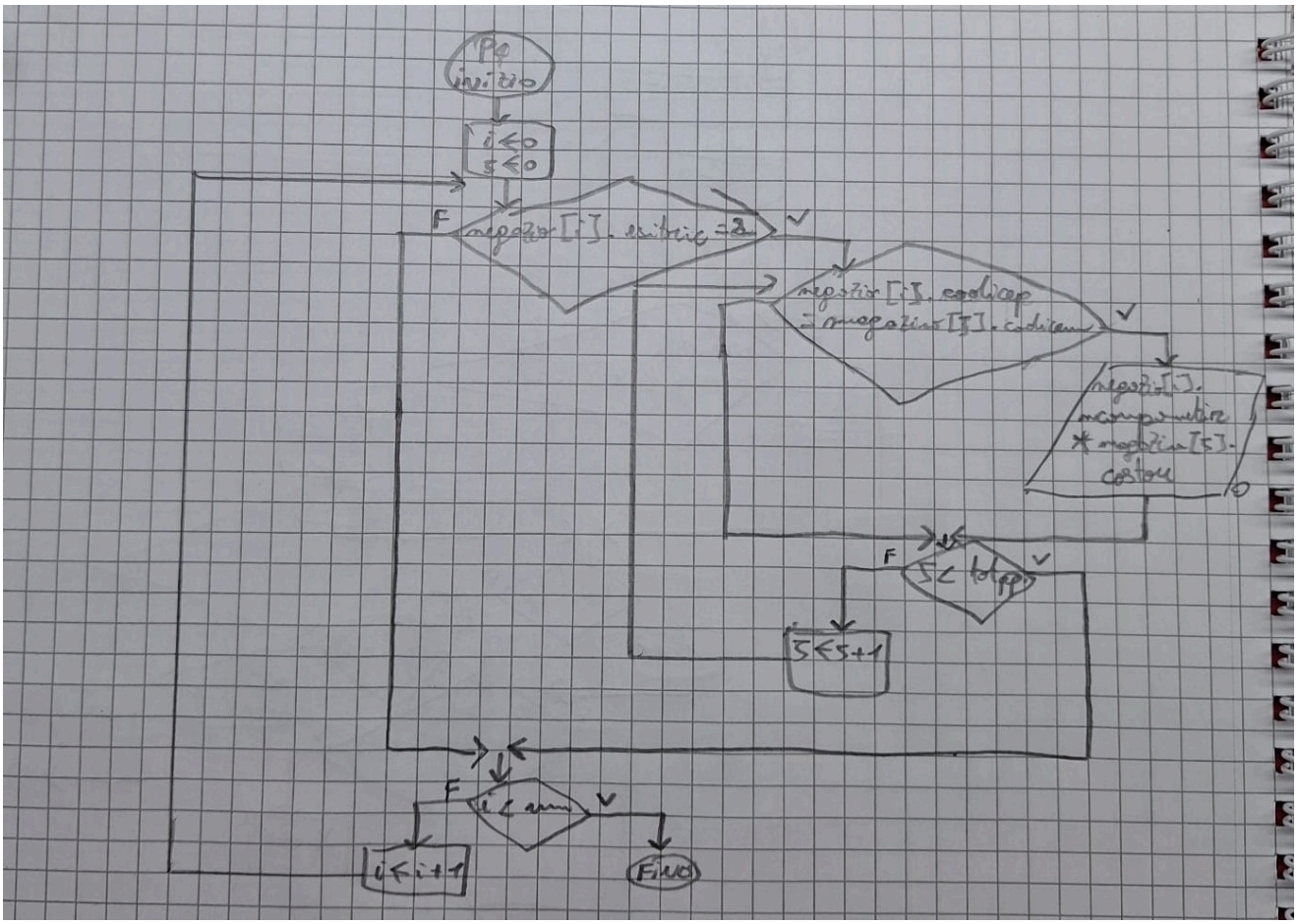
FLOWCHART:

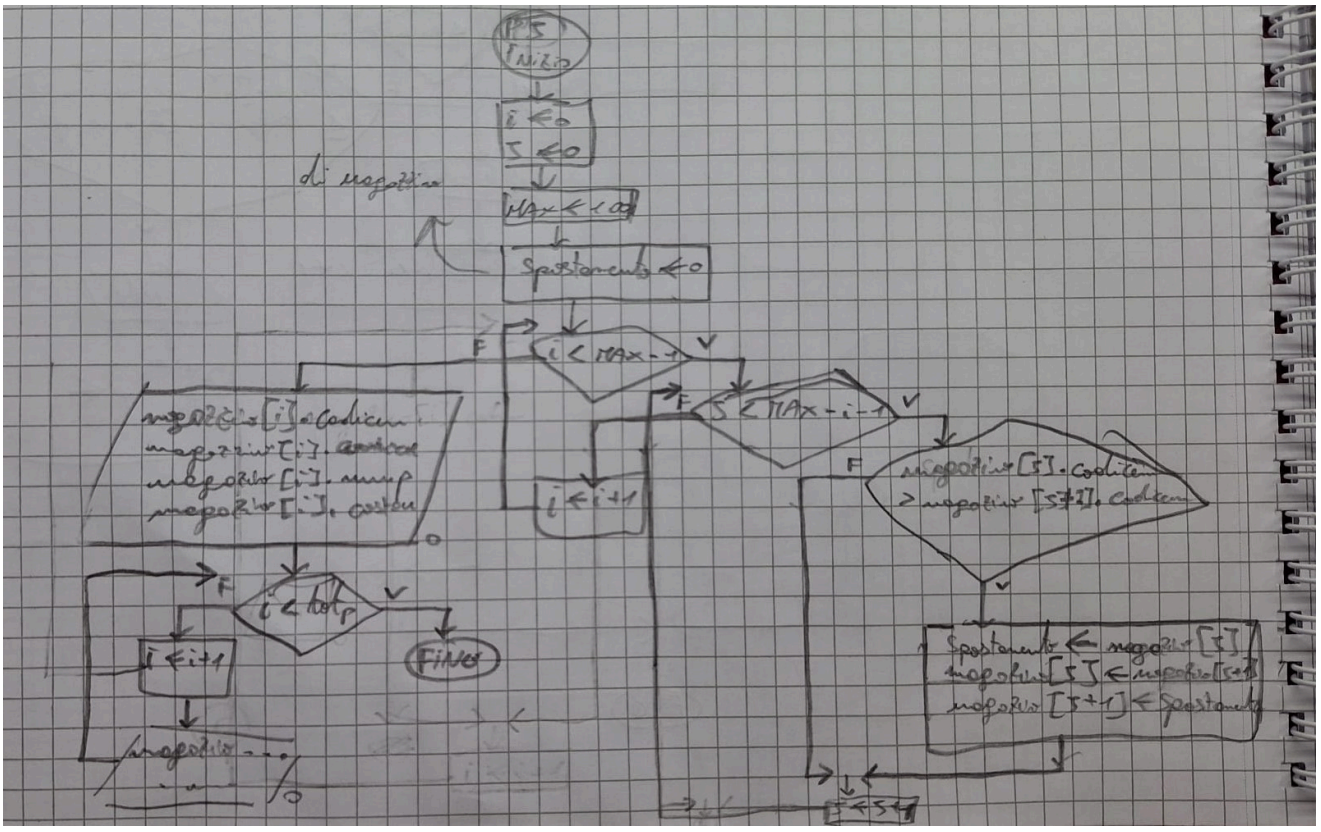
3) FLOW CHART


P1
INIZIO
STRINGA ← 100
MAXC ← 5
i ← 0
negozio[i].codicem
negozio[i].codicep
negozio[i].codicem AND negozio[i].codicep ≥ 0
negozio[i].codicem
negozio[i].codicep
negozio[i].nomp
negozio[i].nomp
i ← i+1
FINE


P2
INIZIO
totp ← 0
i ← 0
totp
totp > 0
negozio[i].codicem
negozio[i].codicem
≥ 0 ≤ MAXC
negozio[i].codicem
negozio[i].nomp
negozio[i].costop
i ← i+1
FINE


P3 INIZIO
richiesto ← 0
i ← 0
richiesta I
F
richiesta = magazzino[i].codicem
V
flag ← 0
i ← i+1
F
i < totp
V
i ← i+1
F
flag != 0
V
F
richiesta = negozio[i].codicep
V
Negozio non presente O
A
X ← i
F
i < numn
V
i ← i+1
F
negozio[x].acquistati ≤ magazzino[j].nump
V
Negozio O
negozio[x].esitoric ← 1
Accettato O
magazzino[j].nump ← magazzino[j] - negozio[x].numc
negozio[x].esitoric ← 2
A
FINE





(QUESTO BUBBLE SORT E' DA CONTROLLARE)

PUNTO D

CODICE:

```c
/*************************************************************************
***

Bricolage

*************************************************************************
***/
#include <stdio.h>
#include <string.h>
#define STRINGA 100
#define MAX 100
#define MAXC 9999

//strutture
typedef struct{
    int codicen;
    int codicep;
    char nomep[STRINGA];
    int ncomponentir;
    short esito_ric;
}negozio_t;

typedef struct{
    int codicem;
    char nomem[STRINGA];
    int nump;
    float costou;
}magazzino_t;

//prototipi delle funzioni
int inserimento_negozi(negozio_t negozio[]);
int inserimento_magazzino(magazzino_t magazzino[]);
void richiesta_prodotto(negozio_t negozio[], magazzino_t magazzino[], int
numn, int totp);
void costocomplessivo(negozio_t negozio[],magazzino_t magazzino[], int numn,
int totp);
void ordinamento(magazzino_t magazzino[],int totp);

//main
int main()
{
    negozio_t n[MAX];
    magazzino_t m[MAX];
    int numero_negozi=0, numero_prodottim=0;
    numero_negozi=inserimento_negozi(n);
    numero_prodottim=inserimento_magazzino(m);
    richiesta_prodotto(n,m,numero_negozi,numero_prodottim);
    costocomplessivo(n,m,numero_negozi,numero_prodottim);
    ordinamento(m,numero_prodottim);
```

```c
    return 0;
}

//implementazione funzioni
int inserimento_negozi(negozio_t negozio[]){
    int numn=0;
    do{
        printf("Inserire il numero dei negozi presenti della sua catena: ");
        scanf("%d", &numn);
    }while(numn<0);
    for(short i=0;i<numn;i++){
        do{
            printf("\nInserire il codice del negozio: ");
            scanf("%d", &negozio[i].codicen);
        }while(negozio[i].codicen<0 || negozio[i].codicen>MAXC);

        do{
            printf("\nInserire il codice del prodotto richiesto: ");
            scanf("%d", &negozio[i].codicep);
        }while(negozio[i].codicen<0 || negozio[i].codicen>MAXC);

        printf("\nInserire il nome del prodotto: ");
        scanf("%s", negozio[i].nomep);

        printf("\nInserire il numero dei componenti richiesti: ");
        scanf("%d", &negozio[i].ncomponentir);
    }
}

int inserimento_magazzino(magazzino_t magazzino[]){
    int totp=0;
    do{
        printf("\nInserire il numero delle classi di prodotti presenti nel
magazzino: ");
        scanf("%d", &totp);
    }while(totp<0);

    for(short i=0;i<totp;i++){
        do{
            printf("\nInserire il codice del materiale: ");
            scanf("%d", &magazzino[i].codicem);
        }while(magazzino[i].codicem<0 || magazzino[i].codicem>MAXC);

        printf("\nInserire il nome del prodotto: ");
        scanf("%s", magazzino[i].nomem);

        printf("\nInserire il numero dei pezzi presenti: ");
        scanf("%d", &magazzino[i].nump);
```

```
        printf("\nInserire il costo unitario del prodotto: ");
        scanf("%f", &magazzino[i].costou);
    }
}

void richiesta_prodotto(negozio_t negozio[], magazzino_t magazzino[], int
numn, int totp){
    int richiesta=0;
    short flag=1,j=0,x=0;
    printf("\nInserire il codice del prodotto che si vuole cercare: ");
    scanf("%d", &richiesta);
    for(short i=0;i<totp;i++){
        if(richiesta==magazzino[i].codicem){
            flag=0;
            j=i;
        }
    }
    if(flag!=0) printf("\nRichiesta negata, prodotto non presente");
    for(short i=0;i<numn;i++){
        if(richiesta==negozio[i].codicep) x=i;
    }
    if(negozio[x].ncomponentir<= magazzino[j].nump){
        printf("\nRichiesta accettata");
        negozio[x].esito_ric=2;
        magazzino[j].nump=magazzino[j].nump - negozio[x].ncomponentir;
    }
    else{
        printf("\nRichiesta negata");
        negozio[x].esito_ric=1;
    }
}

void costocomplessivo(negozio_t negozio[], magazzino_t magazzino[], int
numn, int totp){
    for(short i=0;i<numn;i++){
        if(negozio[i].esito_ric==2){
            for(short j=0;j<totp;j++){
               if(negozio[i].codicep==magazzino[j].codicem) printf("\nIl
costo complessivo del prodotto ordinato è di: %f", magazzino[j].costou *
negozio[i].ncomponentir);
            }
        }
    }
}

void ordinamento(magazzino_t magazzino[],int totp){
    magazzino_t spostamento; //variabile temporanea
    int p=totp;
    int scambio=0;
    do{
```

```
        scambio=0;
        for(short i=0;i<(p-1);i++){
            if(magazzino[i].codicem>magazzino[i+1].codicem){
                spostamento=magazzino[i];
                magazzino[i]=magazzino[i+1];
                magazzino[i+1]=spostamento;
                scambio=1;
            }
        }
        p=p-1;
    }while(scambio && p>0);


    //visualizzazione nuovo ordine
    for(short i=0;i<totp;i++){
        printf("\nIl nuovo magazzino ordinato per codice e': ");
        printf("\n%d", magazzino[i].codicem);
        printf("\n%s", magazzino[i].nomem);
        printf("\n%d", magazzino[i].nump);
        printf("\n%f", magazzino[i].costou);
        printf("\n");
    }
}
```